



KERNA



KERNA RBAC MODEL (FOUNDATION)

Core Roles

FOUNDER
MANAGEMENT
SALES
DEV
DESIGN
RESEARCH
CLIENT



PERMISSION STRATEGY

We do NOT hardcode permissions into routes.

We implement:

- 1 Role-Based Access (RBAC)
 - 2 Project-Scoped Access
 - 3 Module-Scoped Access
 - 4 Action-Level Access
 - 5 Immutable Audit Enforcement
-



PERMISSION STRUCTURE

Sneha should implement a permission registry like this:

type Permission =

"LEADS_VIEW"
"LEADS_EDIT"
"QUOTATION_CREATE"
"QUOTATION_EDIT"
"PROJECT_VIEW"
"PROJECT_EDIT"
"MODULE_VIEW"
"MODULE_EDIT"
"PAYMENT_VIEW"
"PAYMENT_EDIT"
"VAULT_VIEW"
"VAULT_DOWNLOAD"
"CREDENTIAL_VIEW"
"ANALYTICS_VIEW"
"SYSTEM_EDIT"

ROLE → PERMISSION MATRIX

FOUNDER

- All permissions
 - EXCEPT: cannot override client approval state
 - Cannot modify immutable audit logs
-

MANAGEMENT

- VIEW analytics
 - VIEW projects
 - VIEW payments
 - NO edit permissions
 - NO credential visibility
 - NO system edit
-

SALES

- LEADS_VIEW
 - LEADS_EDIT
 - QUOTATION_CREATE
 - QUOTATION_EDIT (before approval)
 - PROJECT_VIEW (only assigned POC projects)
 - PAYMENT_VIEW (only POC projects)
-

DEV

- PROJECT_VIEW (assigned projects only)
 - MODULE_VIEW (dev modules only)
 - MODULE_EDIT (dev modules only)
 - NO PAYMENT_VIEW
 - NO CREDENTIAL_VIEW
 - NO ANALYTICS
-

DESIGN / RESEARCH

Same as DEV but scoped to design modules.

CLIENT

- PROJECT_VIEW (own projects only)
- MODULE_VIEW
- VAULT_VIEW

- VAULT_DOWNLOAD (if fully paid)
- CREDENTIAL_VIEW
- APPROVAL_ACTION

NO internal data.



ROUTE GUARD FLOW

Sneha should implement guard flow in this order:

Step 1 — Authentication Check

if (!user) redirect("/login")

Step 2 — Role Check

```
if (!roleAllowed(route, user.role))  
  return 403
```

Step 3 — Project Scope Check

If route includes `:projectId`

```
if (!user.hasAccessToProject(projectId))  
  return 403
```

Step 4 — Module Scope Check

If route includes `:moduleId`

```
if (!user.hasAccessToModule(moduleId))  
  return 403
```

Step 5 — Action-Level Check

For POST/PUT/PATCH routes:

```
if (!user.hasPermission(requiredPermission))  
  return 403
```



SPECIAL RULES (KERNA-SPECIFIC)

1 Client Approval Lock

Once client approves a version:

```
if (version.status === "APPROVED") {  
  block edits  
}
```

Only allowed:

- POC can reopen with reason
 - Founder cannot override silently
-

2 Immutable Audit Enforcement

Every mutation must log:

```
{  
  userId,  
  role,  
  entityType,  
  entityId,  
  action,  
  timestamp,  
  previousStateHash,  
  newStateHash  
}
```

No delete.
Only append.

3 Payment-Gated Downloads

Before vault download:

```
if (project.payment_status !== "FULLY_PAID")
```

blockDownload()

Client can preview.
Not download.

4 POC Reassignment Rule

Only Founder can reassign POC.

```
if (action === "REASSIGN_POC" && user.role !== "FOUNDER")  
  return 403
```

5 Dual Deadline Visibility

Internal deadlines visible to:

- Founder
- Dev
- Design
- POC

Client sees only client deadline.

Guard logic:

```
if (user.role === "CLIENT")  
  hideInternalDeadlines()
```



PROJECT ACCESS MODEL

Each project record must include:

```
{  
  projectId,  
  clientId,  
  poId,  
  assignedDevIds: [],  
  assignedDesignIds: [],  
  assignedResearchIds: []  
}
```

Access logic:

```
if (user.role === "DEV")  
  allow if assignedDevIds.includes(user.id)
```



ROUTE GUARD EXAMPLE (Pseudo Code)

```
function routeGuard(req, user) {  
  
  if (!user) throw Unauthorized  
  
  const routeMeta = getRouteMeta(req.path)  
  
  if (!routeMeta.allowedRoles.includes(user.role))  
    throw Forbidden  
  
  if (routeMeta.requiresProjectScope) {  
    if (!user.hasAccessToProject(req.params.projectId))  
      throw Forbidden  
  }  
  
  if (routeMeta.requiredPermission) {  
    if (!user.permissions.includes(routeMeta.requiredPermission))  
      throw Forbidden  
  }  
  
  return next()  
}
```



CLIENT MODE ENFORCEMENT (UI Level)

Mode is NOT route-based.
It's component-based.

```
if (user.role === "CLIENT") {  
  renderComponentsBasedOnMode(client.viewMode)  
}
```

Do NOT fork routes.



ESCALATION AUTOMATION

If POC inactive > 1 day:

Trigger:

```
notifyFounder(projectId, "POC_INACTIVE")
```

If Client inactive > 2 days (pending approval):

```
notifyPOC(projectId, "CLIENT_PENDING")
```



SECURITY BEST PRACTICES

Sneha must:

- Never rely only on frontend checks
- Validate everything server-side
- Mask credential fields
- Encrypt stored passwords
- Use short-lived approval tokens